

CS240 Algorithm Design and Analysis
Spring 2025
Problem Set 3

Due: 11:59 pm, 4.30, 2025

1. Submit your solutions to the course Gradescope.
2. If you want to submit a handwritten version, scan it clearly.
3. Late submissions are not allowed.
4. You are required to follow ShanghaiTech's academic honesty policies. You are allowed to discuss problems with other students, but you must write up your solutions by yourselves. You are not allowed to copy materials from other students or from online or published resources. Violating academic honesty can result in serious penalties.

Problem 1:

For each of the two questions below, decide whether the answer is “Yes,” “No,” “Unknown, because it would resolve the question of whether $\mathcal{P} = \mathcal{NP}$.” And give a brief explanation of your answer.

First let’s recall the Interval Scheduling problem. given a collection of intervals, each with a start time and an end time. The task is to choose the maximum number of intervals such that no two intervals overlap.

Formally, you are given a set of intervals $I = \{(s_1, e_1), (s_2, e_2), \dots, (s_n, e_n)\}$, where each interval (s_i, e_i) has a start time s_i and an end time e_i , and your goal is to select a subset of these intervals such that:

1. No two intervals in the subset overlap, i.e., for any two selected intervals (s_i, e_i) and (s_j, e_j) , we must have either $e_i \leq s_j$ or $e_j \leq s_i$.
2. The number of selected intervals is maximized.

Let’s define the decision version of the Interval Scheduling Problem as follows: Given a collection of intervals on a time-line, and a bound k , does the collection contain a subset of nonoverlapping intervals of size at least k ?

- (a) Is it the case that Decision Interval Scheduling $\leq_P$ Vertex Cover?
- (b) Is it the case that Independent Set $\leq_P$ Decision Interval Scheduling?

Solution:

(a) Yes.

Explain 1: Interval Scheduling is in NP, and anything in NP can be reduced to Vertex Cover.

Explain 2: Based on the reductions that Interval Scheduling $\leq_P$ Independent Set and Independent Set $\leq_P$ Vertex Cover, so the result follows by transitivity.

1. Construct a Graph:

- Each interval in the **Interval Scheduling** problem becomes a vertex in the graph.
- There is an edge between two vertices if their corresponding intervals overlap. Specifically, for two intervals $I_i = (s_i, e_i)$ and $I_j = (s_j, e_j)$, there is an edge between the two vertices if $s_i < e_j$ and $s_j < e_i$, i.e., if the two intervals overlap.

2. Independent Set in the Graph:

- An **Independent Set** in this graph corresponds to a set of intervals in the **Interval Scheduling** problem that do not overlap. This is because, by definition, an Independent Set in a graph has no edges between its vertices, meaning no two intervals in this set overlap.

3. Finding the Maximum Independent Set:

- The goal in **Interval Scheduling** is to select the maximum number of non-overlapping intervals. In the graph, this corresponds to finding the largest **Independent Set** (since the intervals corresponding to the vertices in the Independent Set do not overlap).

Thus, solving the Independent Set problem on this graph gives us a solution to the Interval Scheduling problem, because the largest independent set in the graph corresponds to the largest set of non-overlapping intervals.

(b) This is equivalent to whether $\mathcal{P} = \mathcal{NP}$.

The reduction in (a) shows that if we can solve the Independent Set problem efficiently, we can solve Interval Scheduling as well. However, since Independent Set is an NP-complete problem, the reduction implies that Interval Scheduling is at least as hard as Independent Set, unless $\mathcal{P} = \mathcal{NP}$.

Problem 2:

Given a set C , a collection of subsets of C and an integer $k \geq 1$, the Set-Cover problem asks whether there are at most k subsets from the collection which cover C , i.e., whose union includes all of C . Show that Set-Cover is NP-complete.

Hint: You should follow the steps in lecture 13, page 36.

Solution:

Proof of NP

- Given k subsets, we can check if their union covers all elements in C in polynomial time. This involves verifying each element of C is in the union of the k subsets.
- Since checking the union of k subsets can be done in polynomial time, the verification process is polynomial in the size of the input.

So set cover is NP.

Proof of NP-Complete

Vertex Cover can reduce to Set-Cover problem

- Let the set C be the set of edges E of the graph G . For each vertex $v \in V$, create a subset S_v consisting of all edges incident to v . The collection of subsets is $\{S_v \mid v \in V\}$. Set k remains the same.
- To map Vertex Cover to Set-Cover, each subset S_v corresponds to the edges covered by vertex v . The goal is to find k subsets whose union covers all edges E . This is equivalent to finding k vertices that cover all edges in the graph.
- If the Vertex Cover instance has a solution V' with $|V'| \leq k$, then the corresponding subsets $\{S_v \mid v \in V'\}$ form a solution to the Set-Cover instance, as their union covers all edges. Conversely, if the Set-Cover instance has a solution with k subsets, the corresponding vertices form a solution to the Vertex Cover instance.

So set cover is NP-complete.

Problem 3:

Given a Boolean CNF (conjunctive normal form) formula ϕ and an integer $k \geq 1$, the Stingy-SAT problem asks whether the formula has a satisfying assignment in which at most k variables are set to true. Prove that Stingy-SAT is NP-complete.

Hint: You should follow the steps in lecture 13, page 36.

Solution:

Proof of NP

- Given an assignment of the variables, we can check if the assignment satisfies the CNF formula φ in polynomial time.
- We can also count the number of variables set to true in polynomial time and check if this number is at most k .

So Stingy-SAT is NP

Proof of NP-Complete

- Given a Boolean CNF formula ϕ , we use it both in Stingy-SAT and SAT. Set k to the total number of variables in ϕ . Let n be the number of variables in ϕ . Set $k = n$.
- By setting $k = n$, the Stingy-SAT problem is equivalent to finding a satisfying assignment with at most n variables set to true. Since there are n variables in total, this does not restrict the solution space.
- If the SAT instance ϕ is satisfiable, then there exists an assignment that satisfies ϕ . This assignment will also satisfy the condition of Stingy-SAT with $k = n$. Conversely, if there is a satisfying assignment for the Stingy-SAT instance with $k = n$, this means there is a satisfying assignment for the original SAT instance ϕ .

So Stingy-SAT is NP-complete.

Problem 4:

Consider the following problem about scheduling courses. The problem is defined by 3 sets C , S and R . Here, C represents a set of courses, S represents the available time slots for the courses, and R contains a collection of course requests from students, where each request is a subset of C representing the courses that a particular student wishes to take. The goal is to schedule all the courses without any conflicts.

For example, if $C = \{a, b, c, d\}$, $S = \{1, 2, 3\}$, $R = \{\{a, b, c\}, \{a, b, d\}, \{b, d\}\}$, then it is possible to schedule the courses without conflicts by assigning course a to time slot 1, b to slot 2 and c, d to slot 3. However, if $C = \{a, b, c, d\}$, $S = \{1, 2, 3\}$, $R = \{\{a, b, c\}, \{a, c\}, \{b, c, d\}\}$, then it is not possible to schedule the courses without any conflicts. Prove that the problem of determining whether a conflict-free schedule exists is NP-complete.

Hint: Use a reduction from the 3-COLOR problem. You should follow the steps in lecture 13, page 36.

Solution:

Proof of NP: Given a schedule (a potential solution), we can verify in polynomial time whether it is a valid schedule. For each course in C , we check if it is assigned to a time slot in S and that no two courses in the same request from R are assigned to the same time slot. This can be done in polynomial time, so the problem is in NP.

Proof of NP-complete: We will reduce the 3-COLOR problem to the course scheduling problem. Given an instance of the 3-COLOR problem (a graph G with vertices V and edges E), we construct an instance of the course scheduling problem as follows:

- The set of vertices V can be mapped to course C . If there are n vertices, then there will be n courses need us to schedule
- The colors will be mapped to the time slots. In the example 3 color can mapped to 3 time slots. If there are more colors, correspondingly, there will also be more time slots. As 3-coloring can be reduced to n -coloring, this kind of map ensures the complexity will not decrease.
- Set R of requests is constructed from the edges E . For each edge (u, v) in E , we add a request u, v to R .

Now, a conflict-free schedule for the courses corresponds to a valid 3-coloring of the graph, and vice versa. Therefore, the problem of determining whether a conflict-free schedule exists is NP-complete.

Reading materials

- *Introduction to Algorithms, Third Edition. Chapter 34 NP-Completeness*
- *Algorithm Design. Chapter 8 NP and Computational Intractability*
- *Algorithm Design. Chapter 9 PSPACE: A Class of Problems beyond NP*
- *Algorithm Design. Chapter 10 Extending the Limits of Tractability*

External Exercise and Solution

1. PSPACE Exercise: *Algorithm Design. Page 550*
2. Extending Tractability Exercise: *Algorithm Design. Page 594*
3. Reference Solution: <https://github.com/mathiasuy/Soluciones-Klenberg.git>